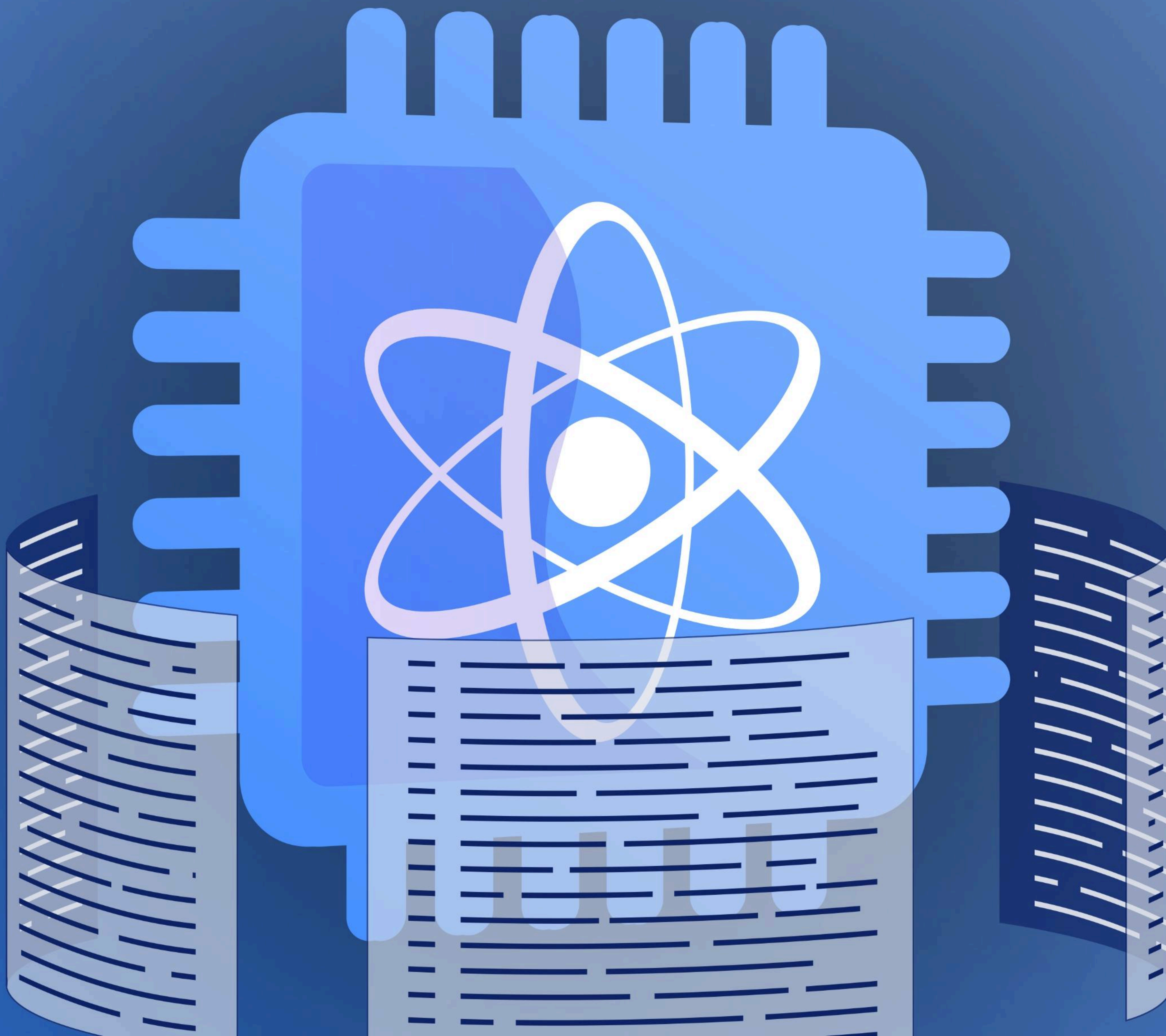


IMPERIO CODERS



MASTER REACT

In Just 31 Days



27. How to update state in React class components?

To update state in a React class component, you can use the **setState** method. This method takes an object or function as an argument, and it will merge the new state with the existing state.

28. How to update state in React functional component?

To update state in a React functional component, you can use the **useState** hook. This hook returns an array containing the current state and a function to update it.

29. How to Differentiate Between State and Props?

State is the internal data of a component that can change and is managed by the component itself. Props are external data passed to a component from its parent component. State can be updated by the component, whereas props cannot be updated by the component.

30. What is Lifting State Up in React?

Lifting state up in React refers to the process of moving state management from a child component to its parent component. This is done to make the state more easily accessible and manageable for the entire component tree.

31. What is an Event in React?

An event in React is a way to respond to user interactions such as clicks, hover, or form submissions.

32. How to Handle Events in React (for both functional and class components)?

In React, events can be handled using the **on** keyword, followed by the event name and a callback function. For example, to handle a button click event, you would use **onClick={handleClick}**. This can be done in both functional and class components.

33. What is the difference between HTML and React event handling?

In HTML, events are handled using event attributes, such as **onclick** or **onchange**. In React, events are handled using the **on** keyword and a callback function. React's event handling system is more powerful and flexible than the traditional HTML event handling system.

34. What are synthetic events in React?

Synthetic events in React are a cross-browser compatible way to handle events. They are used to normalize the behavior of different browser event systems and provide a consistent API for handling events.

35. How to pass parameter to an event handler?

To pass a parameter to an event handler, you can use an arrow function to wrap the event handler. For example: `<button onClick={() => handleClick(parameter)}>`.

36. What are React Hooks?

React Hooks are functions that allow you to use state and other React features in functional components. They were introduced in React 16.8.

37. When were the React Hooks introduced first?

React Hooks were first introduced in React 16.8 in 2019.

38. Can you explain the useState hook with examples?

The useState hook allows you to add state to a functional component. It returns an array with two values: the current state and a function to update it. Here's an example of how to use the useState hook:

```
import React, { useState } from 'react';

function MyComponent() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}

export default MyComponent;
```

39. Can you explain the useEffect hook?

The useEffect hook allows you to run side effects, such as fetching data or updating the DOM, in a functional component. It takes a callback function as its first argument, which is called after the component has rendered. Here's an example of how to use the useEffect hook:

```
import React, { useState, useEffect } from 'react';

function MyComponent() {
  const [data, setData] = useState([]);

  useEffect(() => {
    async function fetchData() {
      const response = await fetch('https://my-api.com/data');
      const json = await response.json();
      setData(json);
    }

    fetchData();
  }, []);

  return (
    <div>
      {data.map(item => (
        <p key={item.id}>{item.name}</p>
      ))}
    </div>
  );
}

export default MyComponent;
```

The useEffect hook is used to run an async function `fetchData` that fetches data from an API and updates the component's state with the result. The empty array `[]` passed as the second argument to `useEffect` means that the effect will only run once, when the component is first rendered.

The useEffect hook allows the component to update its state and re-render in response to changes in some variable. In this case, when the component is first rendered, the data is fetched and the component re-renders with the updated data.

40. When are we using the useMemo hook and why?

useMemo is a hook that allows you to memoize a value. It is used to optimize the performance of a component by only re-computing a value if its dependencies have changed. This can be useful for avoiding expensive calculations or rendering operations. Here's an example of how to use the useMemo hook:

```
import React, { useState, useMemo } from 'react';

function MyComponent() {
  const [a, setA] = useState(1);
  const [b, setB] = useState(1);

  const result = useMemo(() => {
    // This is a costly calculation
    let sum = 0;
    for (let i = 0; i < 10000000; i++) {
      sum += i;
    }
    return sum;
  }, [a, b]);

  return (
    <div>
      <input type="number" value={a} onChange={e => setA(e.target.value)} />
      <input type="number" value={b} onChange={e => setB(e.target.value)} />
      <p>The result is: {result}</p>
    </div>
  );
}

export default MyComponent;
```

In this example, the **useMemo** hook is used to memoize the result of a costly calculation that depends on the values of a and b. The **useMemo** hook takes two arguments: the first is a function that performs the calculation, and the second is an array of dependencies.

The component has two input fields, where the user can set the values of a and b. When either of the inputs change, the component re-renders, but the result is only recalculated if a or b have changed, which is determined by the dependencies array [a,b].

This prevents the costly calculation from being performed every time the component re-renders, improving the performance of the application.

41. What is useRef being used for?

useRef is a hook that allows you to create a reference to a DOM node or a JavaScript object. It can be used to access a DOM node directly, or to store a value that should not cause a re-render when it changes. Here's an example of how to use the useRef hook:

```
import React, { useRef } from 'react';

function MyComponent() {
  const inputRef = useRef(null);

  function handleClick() {
    inputRef.current.value = "Hello World!";
  }

  return (
    <div>
      <input type="text" ref={inputRef}/>
      <button onClick={handleClick}>
        Update Input
      </button>
    </div>
  );
}

export default MyComponent;
```

42. How to create refs?

To create a ref, you can use the **useRef** hook, which returns a ref object. You can then assign this object to a ref attribute on a JSX element.

43. What are forward refs and can you give me a code example for it?

A forward ref is a way to pass a ref through a component to a child component. It allows you to access the ref of a child component from the parent component. Here's an example of how to create a forward ref:

```
import React, { forwardRef } from 'react';

const ChildComponent = forwardRef((props, ref) => {
  return (
    <input type="text" ref={ref}/>
  );
});

function ParentComponent() {
  const inputRef = React.useRef(null);

  function handleClick() {
    inputRef.current.value = "Hello World!";
  }

  return (
    <div>
      <ChildComponent ref={inputRef} />
      <button onClick={handleClick}>
        Update Input
      </button>
    </div>
  );
}

export default ParentComponent;
```

In this example, the **ChildComponent** is defined as a "forward ref" component using the **forwardRef** higher-order component. The **forwardRef** function takes a component and returns a new component that can accept a ref as a prop.

The **ParentComponent** has a ref object **inputRef** that is passed as a prop to the **ChildComponent** using the **ref** attribute. Inside the child component, the **ref** prop is passed as a second argument to the component function, this way is passed to the input element using the ref attribute.

When the button is clicked, it triggers the **handleClick** function, which updates the value of the input field by accessing its **value** property via the ref object. This way, the ref object can be passed through the component hierarchy to access elements deep down in the tree.

44. Can you create your custom React hooks?

Yes, you can create your custom React hooks by following the naming convention "use" and using state and other hooks inside it.

45. What do you need to keep in mind while creating custom React hooks?

When creating custom React hooks, it's important to keep in mind that they should only call other hooks at the top level and not inside loops or conditions.

Also, it's important to make sure that the hook only performs one specific action.

46. What is the Context API in React?

The Context API in React allows you to share data between components without passing props through every level of the component tree. It provides a way for components to access data that is "global" to the component tree, such as a user's authentication status or a theme.

The Context API consists of a Provider component, which provides the data, and a Consumer component, which accesses the data.

47. What is React Router?

React Router is a library for routing in React apps. It allows you to define the different routes in your application and render the appropriate components for each route. This makes it easy to change the displayed content based on the current URL, without having to refresh the page.

48. What are Pure components and what is their purpose?

Pure components are components that only re-render if their props or state have changed. They are optimized for performance, and they can improve the performance of your application by reducing the number of unnecessary re-renders.

Pure components are also known as "functional components" or "stateless components" and they are defined by a function.

49. Can you update the React state directly?

No, you cannot update the React state directly. You should always use the **setState** method to update the state, which will trigger a re-render of the component.

50. What is the purpose of callback function as an argument of **setState()**?

The callback function passed as an argument to **setState** is called after the state has been updated and the component has re-rendered. It can be used to perform any additional actions, such as sending a network request, that depend on the updated state.

51. What is "key" prop and what is the benefit of using it in arrays of elements?

The "key" prop is used to give a unique identifier to each item in an array of elements. When elements are re-rendered, React uses the key to identify which elements have changed, added, or removed. This allows React to update the DOM efficiently, improving the performance of the application.

52. What are the different phases of component lifecycle in React?

The different phases of the component lifecycle in React are:

- Mounting: When a component is being added to the DOM.
- Updating: When a component's props or state change.
- Unmounting: When a component is being removed from the DOM.